# Linear regression workbook

This workbook will walk you through a linear regression example. It will provide familiarity with Jupyter Notebook and Python. Please print (to pdf) a completed version of this workbook for submission with HW #1.

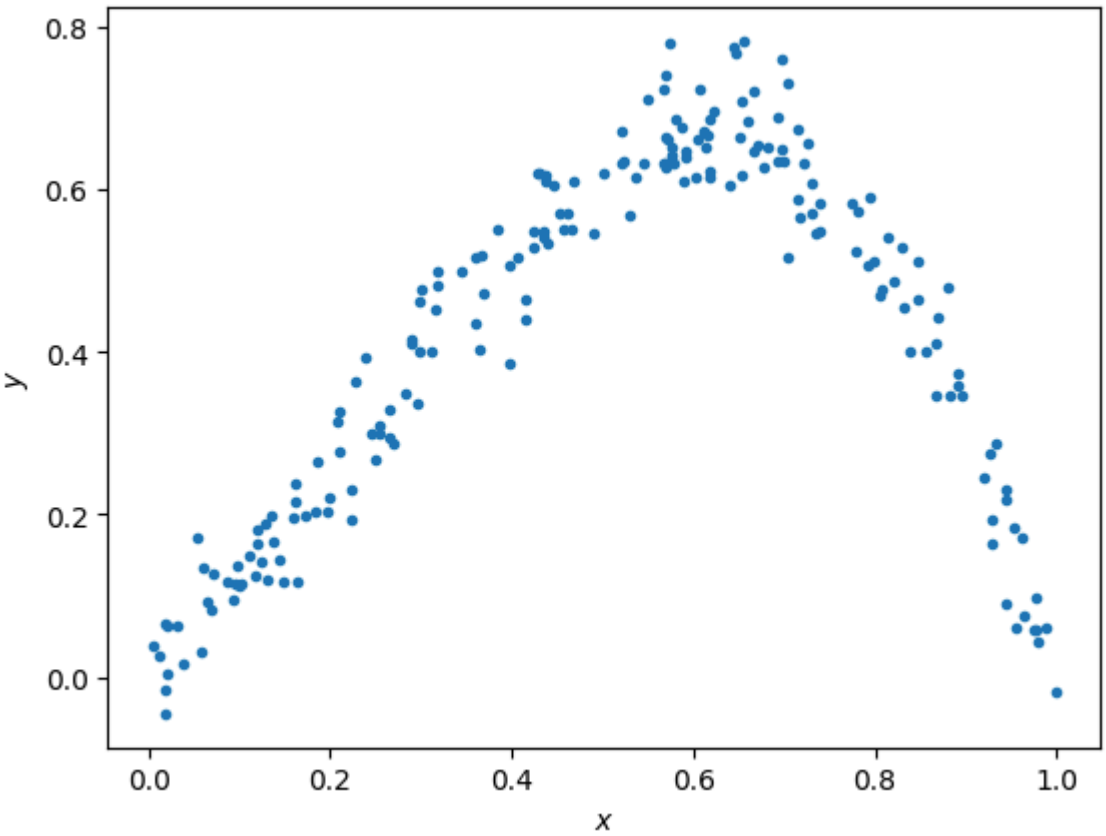
ECE C147/C247, Winter Quarter 2026, Prof. J.C. Kao, TAs: S. Athreya, Y. Chang, S. Dong, K. Pang, M. Turali, L. Zou

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

#allows matlab plots to be generated in line
%matplotlib inline
```

## Data generation

For any example, we first have to generate some appropriate data to use. The following cell generates data according to the model: $y = x + 2x^2 - 3x^3 + \epsilon$

In [2]:
```
np.random.seed(0)  # Sets the random seed.
num_train = 200  # Number of training data points

# Generate the training data
x = np.random.uniform(low=0, high=1, size=(num_train,))
y = x + 2 * x**2 - 3 * x**3 + np.random.normal(loc=0, scale=0.05, size=(n
f = plt.figure()
ax = f.gca() # Get current axis
ax.plot(x, y, ".")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
plt.show()
```




## QUESTIONS:

Write your answers in the markdown cell below this one:

(1) What is the generating distribution of $x$?

(2) What is the distribution of the additive noise $\epsilon$?

## ANSWERS:

(1) The generating distribution of $x$ is a uniform distribution over the interval $[0, 1]$.

(2) The additive noise $\epsilon$ follows a normal (Gaussian) distribution with a mean ($\mu$) of 0 and a standard deviation ($\sigma$) of 0.05.

## Fitting data to the model (5 points)

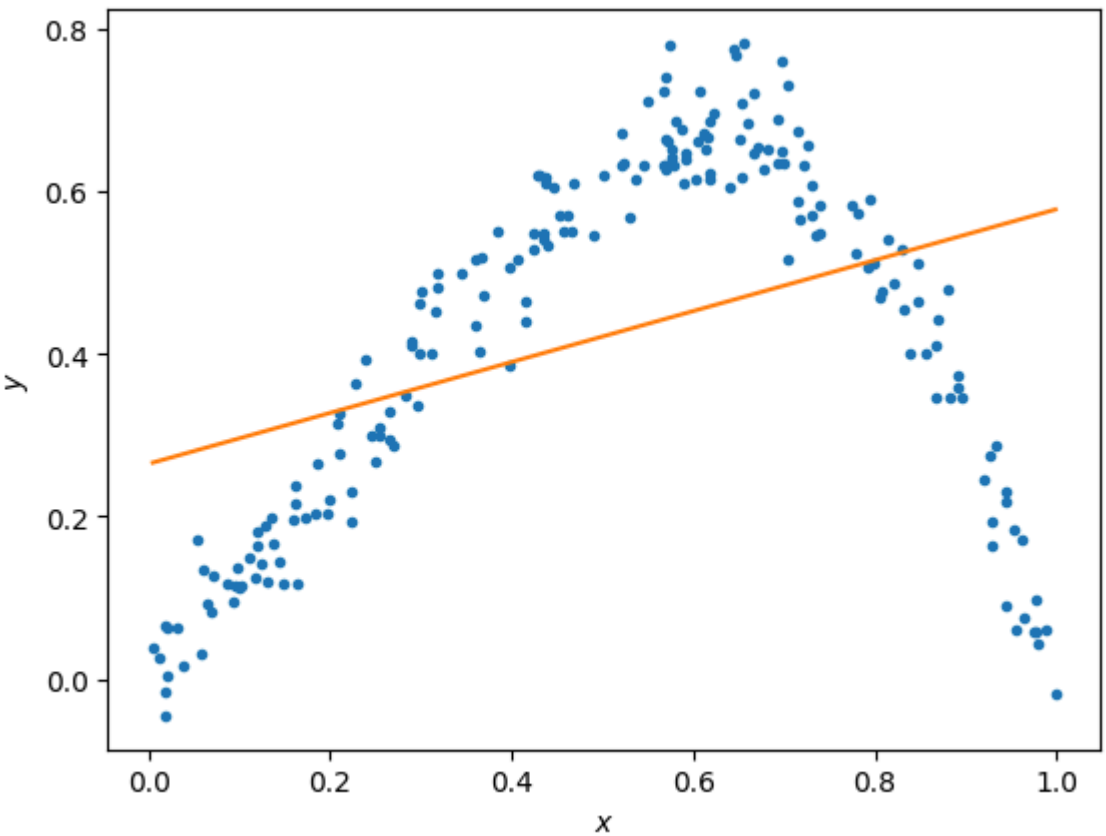
Here, we'll do linear regression to fit the parameters of a model $y = ax + b$.

In [3]:
```
# xhat = (x, 1)
xhat = np.vstack((x, np.ones_like(x)))

# ==================== #
# START YOUR CODE HERE #
# ==================== #
# GOAL: create a variable theta; theta is a numpy array whose elements ar

# theta = np.zeros(2)  # please modify this line
# Calculate theta using the Normal Equation: theta = (X * X^T)^-1 * X * y
```

```
# Here, xhat is X (shape: 2 x N) and y is the target vector (shape: N,)
theta = np.linalg.inv(xhat.dot(xhat.T)).dot(xhat.dot(y))

# ================== #
# END YOUR CODE HERE #
# ================== #
```

In [4]:

```
# Plot the data and your model fit.
f = plt.figure()
ax = f.gca()
ax.plot(x, y, ".")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")

# Plot the regression line
xs = np.linspace(min(x), max(x), 50)
xs = np.vstack((xs, np.ones_like(xs)))
plt.plot(xs[0, :], theta.dot(xs))
plt.show()
```




## QUESTIONS

(1) Does the linear model under- or overfit the data?

(2) How to change the model to improve the fitting?

## ANSWERS

(1) The linear model underfits the data. The data is generated from a cubic function ( $3^{rd}$ order polynomial) and exhibits distinct curvature. A simple linear model (a

straight line) lacks the complexity to capture this non-linear pattern, resulting in high bias.

(2) To improve the fitting, we need to increase the model's complexity to match the data's structure. This can be done by using polynomial regression, which involves adding higher-order feature terms (such as $x^2, x^3$, etc.) to the model to capture the non-linear relationship between $x$ and $y$.

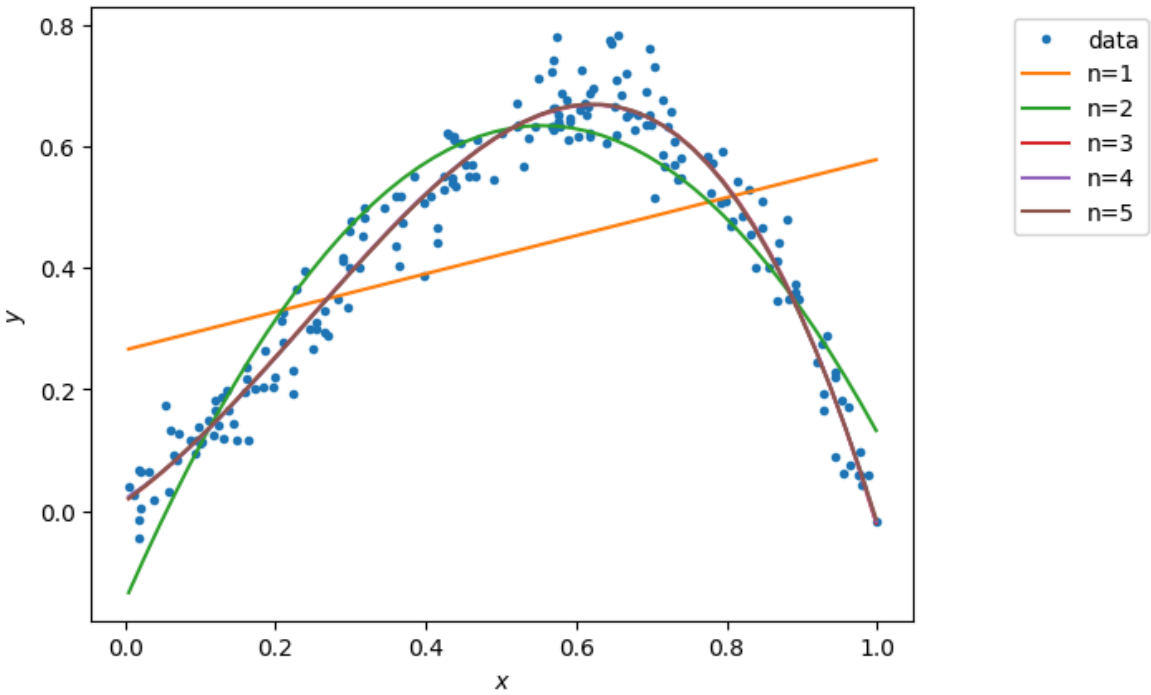
## Fitting data to the model (5 points)

Here, we'll now do regression to polynomial models of orders 1 to 5. Note, the order 1 model is the linear model you prior fit.

In [5]:
```
N = 5
xhats = []
thetas = []

# ==================== #
# START YOUR CODE HERE #
# ==================== #

# GOAL: create a variable thetas.
# thetas is a list, where theta[i] are the model parameters for the polyn
#   i.e., thetas[0] is equivalent to theta above.
#   i.e., thetas[1] should be a length 3 np.array with the coefficients o
#   ... etc.

for i in np.arange(N):
    # Construct the feature matrix for the current polynomial order
    if i == 0:
        # Order 1: Features are [x, 1]
        x_poly = np.vstack((x, np.ones_like(x)))
    else:
        # Order > 1: Add the x^(i+1) term to the features of the previous
        # We retrieve the previous feature matrix from xhats[i-1]
        x_poly = np.vstack((x**(i+1), xhats[i-1]))

    # Store the feature matrix
    xhats.append(x_poly)

    # Calculate theta for the current order using the Normal Equation
    theta_poly = np.linalg.inv(x_poly.dot(x_poly.T)).dot(x_poly.dot(y))

    # Store the parameters
    thetas.append(theta_poly)

# ================== #
# END YOUR CODE HERE #
# ================== #
```

In [6]:
```
# Plot the data
f = plt.figure()
ax = f.gca()
ax.plot(x, y, ".")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
```

```
# Plot the regression lines
plot_xs = []
for i in np.arange(N):
    if i == 0:
        plot_x = np.vstack((np.linspace(min(x), max(x), 50), np.ones(50))
    else:
        plot_x = np.vstack((plot_x[-2] ** (i + 1), plot_x))
    plot_xs.append(plot_x)

for i in np.arange(N):
    ax.plot(plot_xs[i][-2, :], thetas[i].dot(plot_xs[i]))

labels = ["data"]
[labels.append("n={}".format(i + 1)) for i in np.arange(N)]
bbox_to_anchor = (1.3, 1)
lgd = ax.legend(labels, bbox_to_anchor=bbox_to_anchor)
plt.show()
```




## Calculating the training error (5 points)

Here, we'll now calculate the training error of polynomial models of orders 1 to 5.

In [7]:
```
training_errors = []

# ==================== #
# START YOUR CODE HERE #
# ==================== #

# GOAL: create a variable training_errors, a list of 5 elements,
# where training_errors[i] are the training loss for the polynomial fit o
for i in np.arange(N):
    # Predict y values using the training features and learned parameters
    y_pred = thetas[i].dot(xhats[i])

    # Option 1 (Default): Calculate Mean Squared Error (MSE)
    # This is commonly used as it normalizes error by the number of sampl
```

```
    error = np.mean((y - y_pred)**2)

    # Option 2: Calculate Sum of Squared Errors (SSE)
    # If you prefer the raw total error, uncomment the line below:
    # error = np.sum((y - y_pred)**2)

    training_errors.append(error)

# ================== #
# END YOUR CODE HERE #
# ================== #

print("Training errors are: \n", training_errors)
```

```
Training errors are:
 [np.float64(0.04189914875261899), np.float64(0.005860281754804517), np.fl
oat64(0.0022693343891959364), np.float64(0.002268153815360271), np.float64
(0.002267077554312581)]
```

## QUESTIONS

(1) What polynomial has the best training error?

(2) Why is this expected?

## ANSWERS

(1) The polynomial of order 5 has the best (lowest) training error.

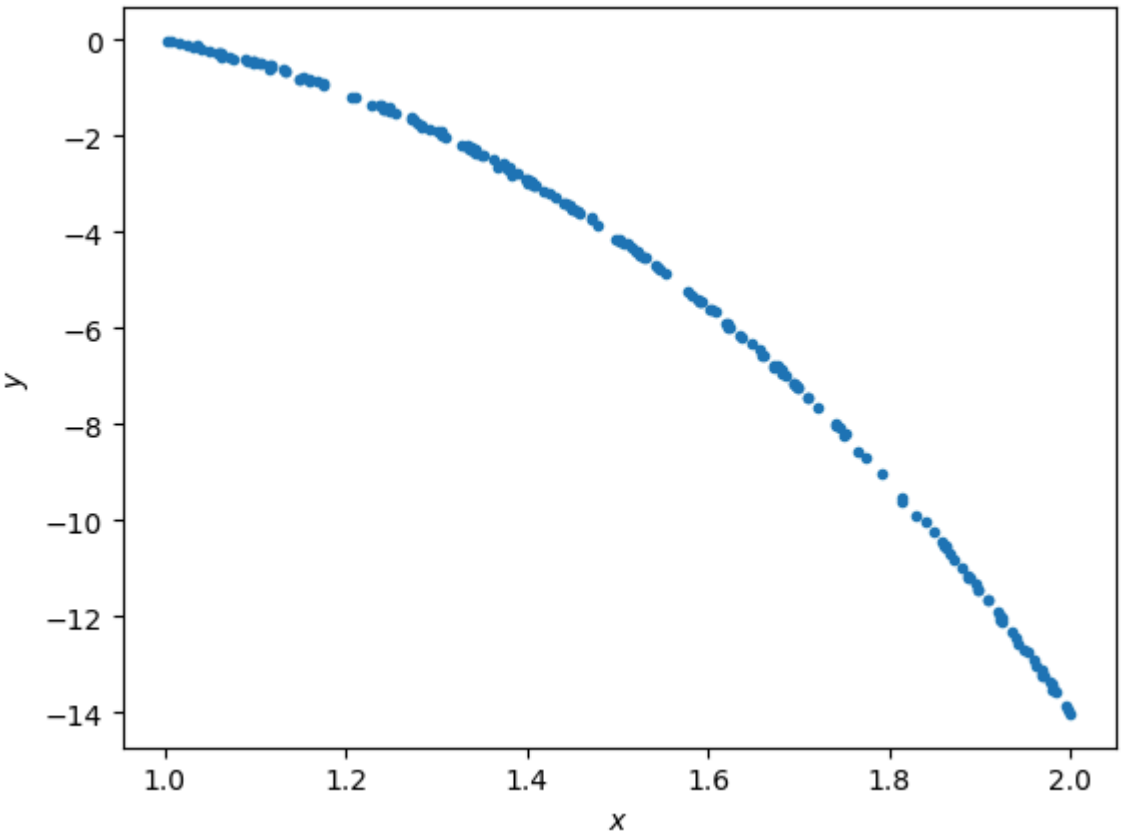
(2) This is expected because the order 5 polynomial is the most complex model among those tested. With more parameters (degrees of freedom), it has the flexibility to pass closer to a larger number of training points, capturing not just the underlying signal but also the random noise in the training set. Therefore, it will always minimize the training error more effectively than lower-order models.

## Generating new samples and testing error (5 points)

Here, we'll now generate new samples and calculate testing error of polynomial models of orders 1 to 5.

In [8]:
```
x = np.random.uniform(low=1, high=2, size=(num_train,))
y = x + 2 * x**2 - 3 * x**3 + np.random.normal(loc=0, scale=0.03, size=(n
f = plt.figure()
ax = f.gca()
ax.plot(x, y, ".")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
```

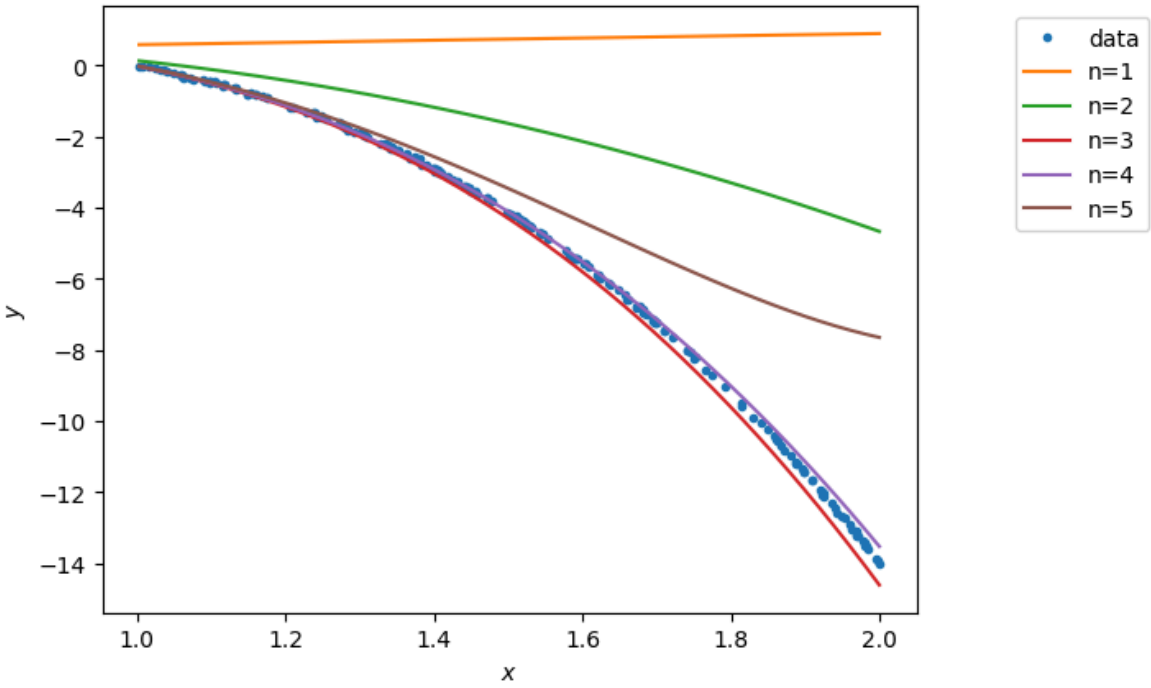
Out[8]:
```
Text(0, 0.5, '$y$')
```




In [9]:
```
xhats = []
for i in np.arange(N):
    if i == 0:
        xhat = np.vstack((x, np.ones_like(x)))
        plot_x = np.vstack((np.linspace(min(x), max(x), 50), np.ones(50))
    else:
        xhat = np.vstack((x ** (i + 1), xhat))
        plot_x = np.vstack((plot_x[-2] ** (i + 1), plot_x))

    xhats.append(xhat)
```

In [10]:
```
# Plot the data
f = plt.figure()
ax = f.gca()
ax.plot(x, y, ".")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")

# Plot the regression lines
plot_xs = []
for i in np.arange(N):
    if i == 0:
        plot_x = np.vstack((np.linspace(min(x), max(x), 50), np.ones(50))
    else:
        plot_x = np.vstack((plot_x[-2] ** (i + 1), plot_x))
    plot_xs.append(plot_x)

for i in np.arange(N):
    ax.plot(plot_xs[i][-2, :], thetas[i].dot(plot_xs[i]))

labels = ["data"]
[labels.append("n={}".format(i + 1)) for i in np.arange(N)]
bbox_to_anchor = (1.3, 1)
```

```
lgd = ax.legend(labels, bbox_to_anchor=bbox_to_anchor)
plt.show()
```




In [11]:
```
testing_errors = []

# ==================== #
# START YOUR CODE HERE #
# ==================== #

# GOAL: create a variable testing_errors, a list of 5 elements,
# where testing_errors[i] are the testing loss for the polynomial fit of
for i in np.arange(N):
    # Predict y values using the TEST features (xhats) and TRAINED parame
    # Note: 'xhats' here has been updated to contain the test data featur
    y_pred = thetas[i].dot(xhats[i])

    # Option 1 (Default): Calculate Mean Squared Error (MSE)
    error = np.mean((y - y_pred)**2)

    # Option 2: Calculate Sum of Squared Errors (SSE)
    # If you used SSE for training, uncomment the line below to match:
    # error = np.sum((y - y_pred)**2)

    testing_errors.append(error)

# ================== #
# END YOUR CODE HERE #
# ================== #

print("Testing errors are: \n", testing_errors)
```

```
Testing errors are:
 [np.float64(54.24631771601226), np.float64(18.91115921753081), np.float64
(0.08693862570381264), np.float64(0.030429668670000486), np.float64(5.9543
56338935129)]
```

## QUESTIONS

(1) What polynomial has the best testing error?

(2) Why polynomial models of orders 5 does not generalize well?

## ANSWERS

(1) Based on the simulation results, the polynomial of order 4 has the best testing error (MSE $\approx$ 0.030), slightly outperforming order 3 (MSE $\approx$ 0.087).

(2) The order 5 polynomial does not generalize well because of overfitting. As observed in the results, the testing error for order 5 increases significantly (to $\approx$ 5.95) compared to order 3 and 4. The model fits the training data (including the noise) too closely. When applying this complex model to the testing data (which is in a new range $x \in [1, 2]$), the high-degree terms cause the predictions to diverge drastically from the true values (poor extrapolation).